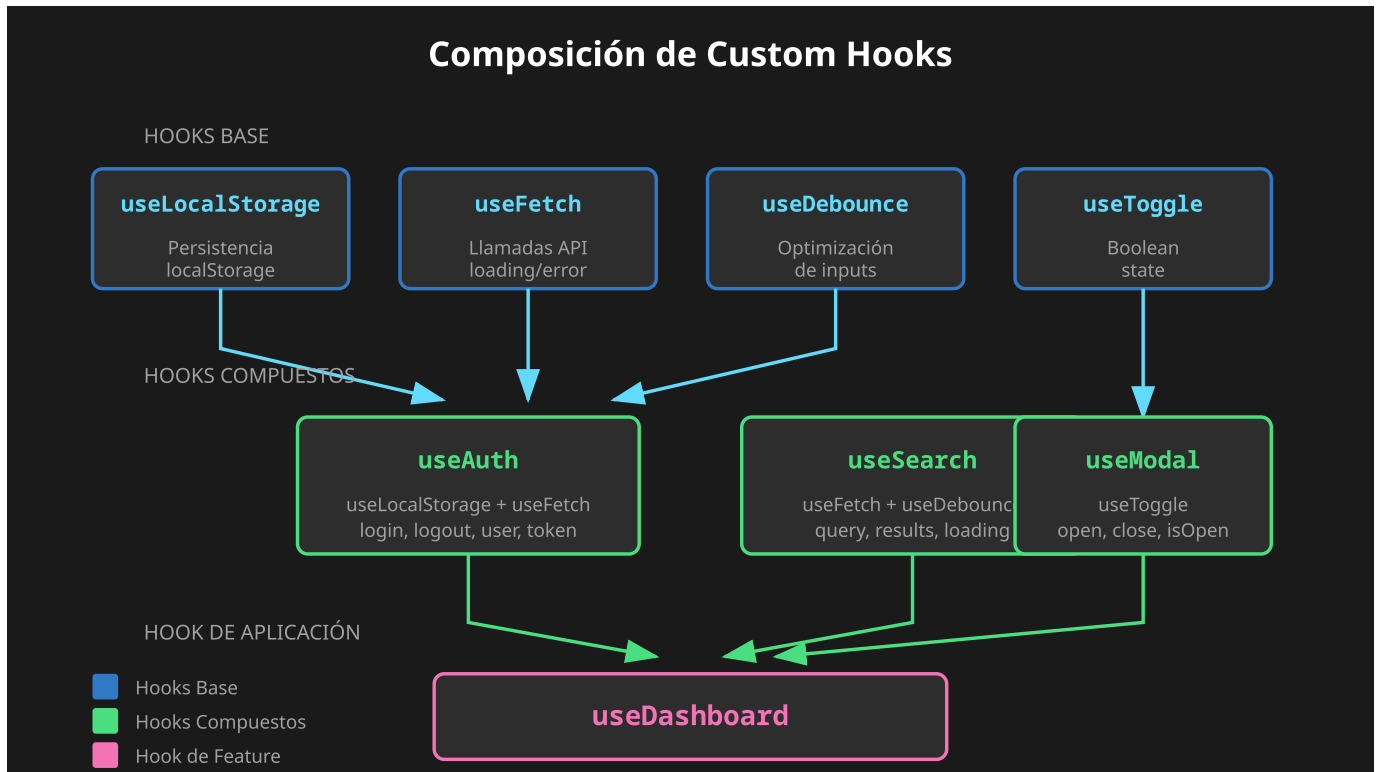


Patrones de Custom Hooks

Objetivos

- Aprender patrones comunes para hooks reutilizables
- Implementar hooks con estado complejo
- Crear hooks con efectos y cleanup
- Entender composición de hooks



1. Patrón: Hook de Estado con Acciones

Hooks que encapsulan estado y proveen acciones específicas.

```
// =====
// QUÉ: Hook useArray para manejar arrays con operaciones CRUD
// PARA: Gestionar listas (todos, carrito, favoritos, etc.)
// IMPACTO: API limpia que encapsula mutaciones inmutables
// =====

import { useState, useCallback } from 'react';

interface UseArrayReturn<T> {
  array: T[];
  set: (newArray: T[]) => void;
  push: (element: T) => void;
  filter: (callback: (element: T) => boolean) => void;
  update: (index: number, newElement: T) => void;
  remove: (index: number) => void;
  clear: () => void;
}
```

```

const useArray = <T>(<initialValue: T[] = []>: UseArrayReturn<T> => {
  const [array, setArray] = useState<T[]>(initialValue);

  const push = useCallback((element: T) => {
    setArray((prev) => [...prev, element]);
  }, []);

  const filter = useCallback((callback: (element: T) => boolean) => {
    setArray((prev) => prev.filter(callback));
  }, []);

  const update = useCallback((index: number, newElement: T) => {
    setArray((prev) => {
      const copy = [...prev];
      copy[index] = newElement;
      return copy;
    });
  }, []);

  const remove = useCallback((index: number) => {
    setArray((prev) => prev.filter((_, i) => i !== index));
  }, []);

  const clear = useCallback(() => {
    setArray([]);
  }, []);

  return { array, set: setArray, push, filter, update, remove, clear };
};

export { useArray };
export type { UseArrayReturn };

```

Uso del Patrón

```

interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

const TodoList: React.FC = () => {
  const todos = useArray<Todo>([]);
  const [inputValue, setInputValue] = useState('');

  const addTodo = () => {
    if (!inputValue.trim()) return;
    todos.push({
      id: Date.now(),
      text: inputValue,
      completed: false,
    });
    setInputValue('');
  };
};

```

```

const toggleTodo = (index: number) => {
  const todo = todos.array[index];
  todos.update(index, { ...todo, completed: !todo.completed });
};

return (
  <div>
    <input
      value={inputValue}
      onChange={(e) => setInputValue(e.target.value)}
      onKeyDown={(e) => e.key === 'Enter' && addTodo()}
    />
    <button onClick={addTodo}>Agregar</button>

    <ul>
      {todos.array.map((todo, index) => (
        <li key={todo.id}>
          <span
            style={{
              textDecoration: todo.completed ? 'line-through' : 'none',
            }}
            onClick={() => toggleTodo(index)}>
            {todo.text}
          </span>
          <button onClick={() => todos.remove(index)}>✖</button>
        </li>
      ))}
    </ul>

    <button onClick={() => todos.filter((t) => !t.completed)}>
      Limpiar completados
    </button>
    <button onClick={todos.clear}>Limpiar todo</button>
  </div>
);
};

```

2. Patrón: Hook con Efectos

Hooks que encapsulan efectos secundarios con cleanup.

```

// =====
// QUÉ: Hook useDebounce para retrasar actualizaciones
// PARA: Optimizar búsquedas, filtros y llamadas a API
// IMPACTO: Reduce llamadas innecesarias mientras el usuario escribe
// =====

import { useState, useEffect } from 'react';

const useDebounce = <T>(value: T, delay: number = 500): T => {
  const [debouncedValue, setDebouncedValue] = useState<T>(value);

  useEffect(() => {
    // Crear timer que actualiza después del delay

```

```

const timer = setTimeout(() => {
  setDebounceValue(value);
}, delay);

// Cleanup: cancelar timer si value cambia antes del delay
return () => {
  clearTimeout(timer);
};
}, [value, delay]);

return debounceValue;
};

export { useDebounce };

```

Uso para Búsqueda

```

const SearchComponent: React.FC = () => {
  const [searchTerm, setSearchTerm] = useState('');
  const debounceSearchTerm = useDebounce(searchTerm, 300);

  // Este efecto solo se ejecuta 300ms después de dejar de escribir
  useEffect(() => {
    if (debounceSearchTerm) {
      console.log('Buscando:', debounceSearchTerm);
      // fetch(`/api/search?q=${debounceSearchTerm}`) ...
    }
  }, [debounceSearchTerm]);

  return (
    <input
      value={searchTerm}
      onChange={(e) => setSearchTerm(e.target.value)}
      placeholder="Buscar..."
    />
  );
};

```

3. Patrón: Hook de Fetch con Estados

Hook completo para fetching con loading, error y data.

```

// =====
// QUÉ: Hook useFetch para llamadas HTTP
// PARA: Manejar estados de carga, error y datos de forma consistente
// IMPACTO: Elimina boilerplate repetitivo en cada componente
// =====

import { useState, useEffect, useCallback } from 'react';

interface UseFetchState<T> {
  data: T | null;
  loading: boolean;
  error: Error | null;
}

```

```

}

interface UseFetchReturn<T> extends UseFetchState<T> {
  refetch: () => Promise<void>;
}

const useFetch = <T>(url: string): UseFetchReturn<T> => {
  const [state, setState] = useState<UseFetchState<T>>({
    data: null,
    loading: true,
    error: null,
  }));

  const fetchData = useCallback(async () => {
    setState((prev) => ({ ...prev, loading: true, error: null }));

    try {
      const response = await fetch(url);

      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }

      const data = await response.json();
      setState({ data, loading: false, error: null });
    } catch (error) {
      setState({
        data: null,
        loading: false,
        error: error instanceof Error ? error : new Error('Error desconocido'),
      });
    }
  }, [url]);

  useEffect(() => {
    fetchData();
  }, [fetchData]);

  return { ...state, refetch: fetchData };
};

export { useFetch };
export type { UseFetchReturn };

```

Versión Mejorada con Cancelación

```

import { useState, useEffect, useCallback, useRef } from 'react';

interface UseFetchOptions {
  enabled?: boolean; // Permite deshabilitar el fetch
}

const useFetchAdvanced = <T>(  
  url: string,  
  options: UseFetchOptions = {},  


```

```

): UseFetchReturn<T> => {
  const { enabled = true } = options;
  const abortControllerRef = useRef<AbortController | null>(null);

  const [state, setState] = useState<UseFetchState<T>>({
    data: null,
    loading: enabled,
    error: null,
  });

  const fetchData = useCallback(async () => {
    // Cancelar request anterior si existe
    if (abortControllerRef.current) {
      abortControllerRef.current.abort();
    }

    // Crear nuevo AbortController
    abortControllerRef.current = new AbortController();
    const { signal } = abortControllerRef.current;

    setState((prev) => ({ ...prev, loading: true, error: null }));

    try {
      const response = await fetch(url, { signal });

      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }

      const data = await response.json();
      setState({ data, loading: false, error: null });
    } catch (error) {
      // Ignorar errores de cancelación
      if (error instanceof Error && error.name === 'AbortError') {
        return;
      }
      setState({
        data: null,
        loading: false,
        error: error instanceof Error ? error : new Error('Error desconocido'),
      });
    }
  }, [url]);

  useEffect(() => {
    if (enabled) {
      fetchData();
    }

    // Cleanup: cancelar request si el componente se desmonta
    return () => {
      if (abortControllerRef.current) {
        abortControllerRef.current.abort();
      }
    };
  }, [fetchData, enabled]);

```

```

    return { ...state, refetch: fetchData };
  };

export { useFetchAdvanced };

```

Uso del Hook Fetch

```

interface User {
  id: number;
  name: string;
  email: string;
}

const UserProfile: React.FC<{ userId: number }> = ({ userId }) => {
  const {
    data: user,
    loading,
    error,
    refetch,
  } = useFetch<User>(`https://jsonplaceholder.typicode.com/users/${userId}`);

  if (loading) return <p>Cargando...</p>;
  if (error) return <p>Error: {error.message}</p>;
  if (!user) return <p>No se encontró usuario</p>;

  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.email}</p>
      <button onClick={refetch}>Actualizar</button>
    </div>
  );
};

```

4. Patrón: Hook de Formulario

Hook para manejar formularios con validación.

```

// =====
// QUÉ: Hook useForm para formularios con validación
// PARA: Manejar estado, errores, touched y submit de forms
// IMPACTO: Formularios robustos sin dependencias externas
// =====

import { useState, useCallback, ChangeEvent, FormEvent } from 'react';

type ValidationRule<T> = (value: T) => string | null;

interface UseFormOptions<T extends Record<string, unknown>> {
  initialValues: T;
  validations?: Partial<Record<keyof T, ValidationRule<T[keyof T]>>>;
  onSubmit: (values: T) => void | Promise<void>;
}

```

```

interface UseFormReturn<T extends Record<string, unknown>> {
  values: T;
  errors: Partial<Record<keyof T, string>>;
  touched: Partial<Record<keyof T, boolean>>;
  isSubmitting: boolean;
  isValid: boolean;
  handleChange: (
    e: ChangeEvent<HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement>,
  ) => void;
  handleBlur: (
    e: ChangeEvent<HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement>,
  ) => void;
  handleSubmit: (e: FormEvent) => Promise<void>;
  setFieldValue: (field: keyof T, value: T[keyof T]) => void;
  reset: () => void;
}

const useForm = <T extends Record<string, unknown>>(<
  options: UseFormOptions<T>,
): UseFormReturn<T> => {
  const { initialValues, validations = {}, onSubmit } = options;

  const [values, setValues] = useState<T>(initialValues);
  const [errors, setErrors] = useState<Partial<Record<keyof T, string>>>({});
  const [touched, setTouched] = useState<Partial<Record<keyof T, boolean>>>({});
  const [isSubmitting, setIsSubmitting] = useState(false);

  // Validar un campo específico
  const validateField = useCallback(
    (field: keyof T, value: T[keyof T]): string | null => {
      const validation = validations[field];
      if (validation) {
        return validation(value);
      }
      return null;
    },
    [validations],
  );

  // Validar todos los campos
  const validateAll = useCallback(()>: boolean => {
    const newErrors: Partial<Record<keyof T, string>> = {};
    let isValid = true;

    for (const field of Object.keys(values) as Array<keyof T>) {
      const error = validateField(field, values[field]);
      if (error) {
        newErrors[field] = error;
        isValid = false;
      }
    }

    setErrors(newErrors);
    return isValid;
  }, [values, validateField]);

```



```

const handleChange = useCallback(
  (
    e: ChangeEvent<
      HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement
    >,
  ) => {
    const { name, value, type } = e.target;
    const fieldValue =
      type === 'checkbox' ? (e.target as HTMLInputElement).checked : value;

    setValues((prev) => ({ ...prev, [name]: fieldValue }));

    // Validar mientras escribe si ya tocó el campo
    if (touched[name as keyof T]) {
      const error = validateField(name as keyof T, fieldValue as T[keyof T]);
      setErrors((prev) => ({ ...prev, [name]: error ?? undefined }));
    }
  },
  [touched, validateField],
);

const handleBlur = useCallback(
  (
    e: ChangeEvent<
      HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement
    >,
  ) => {
    const { name, value } = e.target;
    setTouched((prev) => ({ ...prev, [name]: true }));

    const error = validateField(name as keyof T, value as T[keyof T]);
    setErrors((prev) => ({ ...prev, [name]: error ?? undefined }));
  },
  [validateField],
);

const handleSubmit = useCallback(
  async (e: FormEvent) => {
    e.preventDefault();

    // Marcar todos como tocados
    const allTouched = Object.keys(values).reduce(
      (acc, key) => {
        acc[key as keyof T] = true;
        return acc;
      },
      {} as Record<keyof T, boolean>,
    );
    setTouched(allTouched);

    // Validar todo
    if (!validateAll()) {
      return;
    }
  }
);

```

```

        setIsSubmitting(true);
      try {
        await onSubmit(values);
      } finally {
        setIsSubmitting(false);
      }
    },
    [values, validateAll, onSubmit],
  );

  const setFieldValue = useCallback((field: keyof T, value: T[keyof T]) => {
    setValues((prev) => ({ ...prev, [field]: value }));
  }, []);

  const reset = useCallback(() => {
    setValues(initialValues);
    setErrors({});
    setTouched({});
    setIsSubmitting(false);
  }, [initialValues]);

  const isValid = Object.values(errors).every((error) => !error);

  return {
    values,
    errors,
    touched,
    isSubmitting,
    isValid,
    handleChange,
    handleBlur,
    handleSubmit,
    setFieldValue,
    reset,
  };
};

export { useForm };
export type { UseFormReturn, UseFormOptions, ValidationRule };

```

Uso del Hook Form

```

interface LoginForm {
  email: string;
  password: string;
  remember: boolean;
}

const LoginPage: React.FC = () => {
  const form = useForm<LoginForm>({
    initialValues: {
      email: '',
      password: '',
      remember: false,
    },
  },

```

```

validations: {
  email: (value) => {
    if (!value) return 'El email es requerido';
    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value as string)) {
      return 'Email inválido';
    }
    return null;
  },
  password: (value) => {
    if (!value) return 'La contraseña es requerida';
    if ((value as string).length < 6) {
      return 'Mínimo 6 caracteres';
    }
    return null;
  },
},
onSubmit: async (values) => {
  console.log('Login:', values);
  // await loginUser(values);
},
});

return (
  <form onSubmit={form.handleSubmit}>
    <div>
      <input
        name="email"
        type="email"
        value={form.values.email}
        onChange={form.handleChange}
        onBlur={form.handleBlur}
        placeholder="Email"
      />
      {form.touched.email && form.errors.email && (
        <span className="error">{form.errors.email}</span>
      )}
    </div>

    <div>
      <input
        name="password"
        type="password"
        value={form.values.password}
        onChange={form.handleChange}
        onBlur={form.handleBlur}
        placeholder="Contraseña"
      />
      {form.touched.password && form.errors.password && (
        <span className="error">{form.errors.password}</span>
      )}
    </div>

    <label>
      <input
        name="remember"
        type="checkbox"

```

```

        checked={form.values.remember}
        onChange={form.handleChange}
      />
      Recordarme
    </label>

    <button
      type="submit"
      disabled={form.isSubmitting}>
      {form.isSubmitting ? 'Cargando...' : 'Iniciar sesión'}
    </button>
  </form>
);
};

```

5. Patrón: Composición de Hooks

Hooks que usan otros hooks internamente.

```

// =====
// QUÉ: Hook useAuth que compone useLocalStorage + estado
// PARA: Manejar autenticación con persistencia automática
// IMPACTO: Lógica de auth encapsulada y reutilizable
// =====

import { useMemo } from 'react';

// Hooks base
const useLocalStorage = <T>(key: string, initialValue: T) => {
  const [storedValue, setStoredValue] = useState<T>(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch {
      return initialValue;
    }
  });

  const setValue = useCallback(
    (value: T | ((prev: T) => T)) => {
      const valueToStore =
        value instanceof Function ? value(storedValue) : value;
      setStoredValue(valueToStore);
      window.localStorage.setItem(key, JSON.stringify(valueToStore));
    },
    [key, storedValue],
  );

  return [storedValue, setValue] as const;
};

// Hook compuesto que usa useLocalStorage + useFetch
interface User {
  id: number;
  name: string;
}

```

```

    email: string;
  }

interface AuthState {
  user: User | null;
  token: string | null;
}

interface UseAuthReturn {
  user: User | null;
  token: string | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
}

const useAuth = (): UseAuthReturn => {
  // Usar hook de localStorage para persistencia
  const [authState, setAuthState] = useLocalStorage<AuthState>('auth', {
    user: null,
    token: null,
  });

  const [isLoading, setIsLoading] = useState(false);

  const isAuthenticated = useMemo(() => {
    return authState.token !== null && authState.user !== null;
  }, [authState]);

  const login = useCallback(
    async (email: string, password: string) => {
      setIsLoading(true);
      try {
        // Simular llamada a API
        const response = await fetch('/api/login', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ email, password }),
        });

        if (!response.ok) {
          throw new Error('Credenciales inválidas');
        }

        const { user, token } = await response.json();
        setAuthState({ user, token });
      } finally {
        setIsLoading(false);
      }
    },
    [setAuthState],
  );

  const logout = useCallback(() => {
    setAuthState({ user: null, token: null });
  });

```

```

    // Opcional: llamar endpoint de logout
  }, [setAuthState]);

  return {
    user: authState.user,
    token: authState.token,
    isAuthenticated,
    isLoading,
    login,
    logout,
  };
};

export { useAuth };
export type { UseAuthReturn };

```

6. Patrón: Hook con Reducer

Para estado complejo con múltiples transiciones.

```

// =====
// QUÉ: Hook useCart con useReducer para estado complejo
// PARA: Manejar carrito de compras con múltiples acciones
// IMPACTO: Estado predecible, fácil de testear y debuggear
// =====

import { useReducer, useCallback, useMemo } from 'react';

// Estado del carrito
interface CartItem {
  id: string;
  name: string;
  price: number;
  quantity: number;
}

interface CartState {
  items: CartItem[];
  isOpen: boolean;
}

// Acciones discriminadas
type CartAction =
  | { type: 'ADD_ITEM'; payload: Omit<CartItem, 'quantity'> }
  | { type: 'REMOVE_ITEM'; payload: string }
  | { type: 'UPDATE_QUANTITY'; payload: { id: string; quantity: number } }
  | { type: 'CLEAR_CART' }
  | { type: 'TOGGLE_CART' };

// Reducer puro
const cartReducer = (state: CartState, action: CartAction): CartState => {
  switch (action.type) {
    case 'ADD_ITEM': {
      const existingItem = state.items.find(
        (item) => item.id === action.payload.id,

```

```

    );
    if (existingItem) {
      return {
        ...state,
        items: state.items.map((item) =>
          item.id === action.payload.id
            ? { ...item, quantity: item.quantity + 1 }
            : item,
        ),
      };
    }
    return {
      ...state,
      items: [...state.items, { ...action.payload, quantity: 1 }],
    };
  }
  case 'REMOVE_ITEM':
    return {
      ...state,
      items: state.items.filter((item) => item.id !== action.payload),
    };
  case 'UPDATE_QUANTITY':
    return {
      ...state,
      items: state.items.map((item) =>
        item.id === action.payload.id
          ? { ...item, quantity: action.payload.quantity }
          : item,
      ),
    };
  case 'CLEAR_CART':
    return { ...state, items: [] };
  case 'TOGGLE_CART':
    return { ...state, isOpen: !state.isOpen };
  default:
    return state;
}
};

```

// Hook que encapsula el reducer

```

interface UseCartReturn {
  items: CartItem[];
  isOpen: boolean;
  total: number;
  itemCount: number;
  addItem: (item: Omit<CartItem, 'quantity'>) => void;
  removeItem: (id: string) => void;
  updateQuantity: (id: string, quantity: number) => void;
  clearCart: () => void;
  toggleCart: () => void;
}

```

```

const useCart = (): UseCartReturn => {
  const [state, dispatch] = useReducer(cartReducer, {
    items: [],
    isOpen: false,
  });

```

```

});

// Valores calculados con useMemo
const total = useMemo(() => {
  return state.items.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0,
  );
}, [state.items]);

const itemCount = useMemo(() => {
  return state.items.reduce((sum, item) => sum + item.quantity, 0);
}, [state.items]);

// Action creators envueltos en useCallback
const addItem = useCallback((item: Omit<CartItem, 'quantity'>) => {
  dispatch({ type: 'ADD_ITEM', payload: item });
}, []);

const removeItem = useCallback((id: string) => {
  dispatch({ type: 'REMOVE_ITEM', payload: id });
}, []);

const updateQuantity = useCallback((id: string, quantity: number) => {
  dispatch({ type: 'UPDATE_QUANTITY', payload: { id, quantity } });
}, []);

const clearCart = useCallback(() => {
  dispatch({ type: 'CLEAR_CART' });
}, []);

const toggleCart = useCallback(() => {
  dispatch({ type: 'TOGGLE_CART' });
}, []);

return {
  items: state.items,
  isOpen: state.isOpen,
  total,
  itemCount,
  addItem,
  removeItem,
  updateQuantity,
  clearCart,
  toggleCart,
};
};

export { useCart };
export type { UseCartReturn, CartItem };

```

Recursos Adicionales

- [Patterns.dev: Hooks Patterns](https://patterns.dev/hooks-patterns/)
- [Kent C. Dodds: Advanced React Patterns](https://kentcdodds.com/blog/advanced-react-patterns/)

- [useHooks.com](https://usehooks.com)
-

✓ Checklist de Comprensión

- ☐ Conozco el patrón Estado + Acciones
 - ☐ Sé crear hooks con efectos y cleanup
 - ☐ Puedo implementar un hook de fetch completo
 - ☐ Entiendo la composición de hooks
 - ☐ Sé cuándo usar useReducer en un hook
-

Siguiente: [03-ciclo-vida-componentes.md](#)